LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 3 & 4 – FUNCTION & PROCEDURE

NAMA: Aghneo Amar Ahsan

NIM: 109082500118

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Fungsi (*Function*)

Fungsi adalah satu kesatuan instruksi program yang dirancang untuk memproses input (jika ada) dan selalu mengembalikan suatu nilai (*return value*). Fungsi memetakan suatu input menjadi output dengan tipe data tertentu.

Ciri utama dari sebuah fungsi adalah:

- Memiliki deklarasi tipe data pada nilai yang dikembalikannya.
- Wajib memiliki kata kunci return di dalam badan fungsinya.

Karena sifatnya yang menghasilkan nilai, fungsi biasanya digunakan untuk *assignment* nilai ke suatu variabel, menjadi bagian dari operasi aritmatika/ekspresi, atau langsung dicetak (misal di dalam fmt.Println()). Penamaan fungsi umumnya menggunakan kata benda atau kata sifat yang merepresentasikan nilai yang dikembalikan, contohnya: rerata(), nilaiTerbesar(), atau volumeTabung().

2. Prosedur (*Procedure*)

Berbeda dengan fungsi, Prosedur adalah potongan instruksi program yang tidak mengembalikan nilai secara langsung. Prosedur dibuat untuk menghasilkan suatu efek langsung pada program (misalnya mencetak ke layar, mengubah state, atau memodifikasi variabel secara langsung).

Ciri utama dari sebuah prosedur adalah:

- Tidak ada deklarasi tipe nilai kembalian.
- Tidak menggunakan kata kunci return di dalam badan subprogramnya.

Pemanggilan prosedur dilakukan secara mandiri sebagai sebuah instruksi tunggal, bukan disisipkan di dalam ekspresi atau *assignment* variabel. Oleh karena sifatnya yang berupa instruksi kerja, penamaan prosedur umumnya menggunakan kata kerja, seperti: cetak(), hitungRerata(), atau cariNilai().

3. Parameter dan Argumen

Subprogram (baik fungsi maupun prosedur) dapat berkomunikasi dengan program utama atau pemanggilnya menggunakan parameter. Terdapat dua jenis parameter berdasarkan letaknya:

- Parameter Formal: Parameter yang ditulis pada saat deklarasi subprogram. Ini berfungsi sebagai tempat penampung sementara.

- Parameter Aktual (Argumen): Nilai atau variabel nyata yang dikirimkan ke dalam subprogram saat subprogram tersebut dipanggil. Jumlah dan tipe data parameter aktual harus sama persis dengan parameter formalnya.

4. Mekanisme Alokasi Memori (Pass by Value vs Pass by Reference)

Berdasarkan cara alokasi memorinya saat pemanggilan parameter, terdapat dua metode utama:

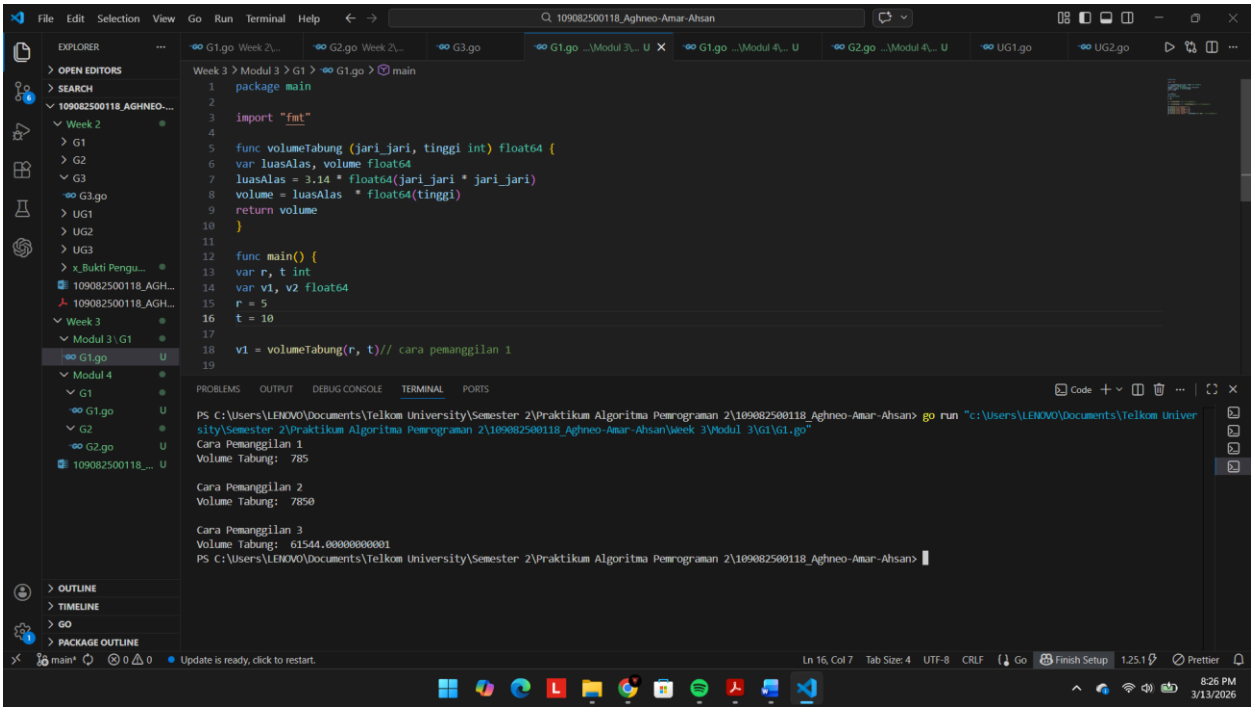
- Pass by Value: Nilai dari parameter aktual *disalin* (di- *copy*) ke parameter formal. Keduanya menempati alamat memori yang berbeda. Jika subprogram mengubah nilai parameter formal, variabel asli pada program utama tidak akan terpengaruh. Metode ini umumnya digunakan secara default pada Fungsi. Pada pseudocode, prosedur menggunakan parameter in untuk konsep ini.
- Pass by Reference (Pointer): Parameter formal bertindak sebagai pointer yang menunjuk langsung ke *alamat memori* dari parameter aktual (menggunakan parameter in/out pada pseudocode). Artinya, setiap perubahan yang terjadi pada parameter formal di dalam subprogram akan langsung merubah nilai variabel asli di program utama. Dalam bahasa GoLang, mekanisme ini diimplementasikan dengan menambahkan tanda asterisk/bintang (*) pada parameter formal dan menggunakan ampersand (&) pada parameter aktual saat pemanggilan. Mekanisme ini umumnya digunakan pada Prosedur agar prosedur seolah-olah dapat "mengembalikan" nilai ke program utama.

B. Guided

1. Volume Tabung

```
package main

import "fmt"

func volumeTabung (jari_jari, tinggi int) float64 {
var luasAlas, volume float64
luasAlas = 3.14 * float64(jari_jari * jari_jari)
volume = luasAlas  * float64(tinggi)
return volume
}

func main() {
var r, t int
var v1, v2 float64
r = 5
t = 10

v1 = volumeTabung(r, t)// cara pemanggilan 1

v2 = volumeTabung(r, t) + volumeTabung(15, t)// cara
pemanggilan 2

fmt.Println("Cara Pemanggilan 1")
fmt.Println("Volume Tabung: ", v1)
fmt.Println("\nCara Pemanggilan 2")
fmt.Println("Volume Tabung: ", v2)
fmt.Println("\nCara Pemanggilan 3")
fmt.Println("Volume Tabung: ", volumeTabung (14, 100)) //
cara pemanggilan 3
}
```

Output :

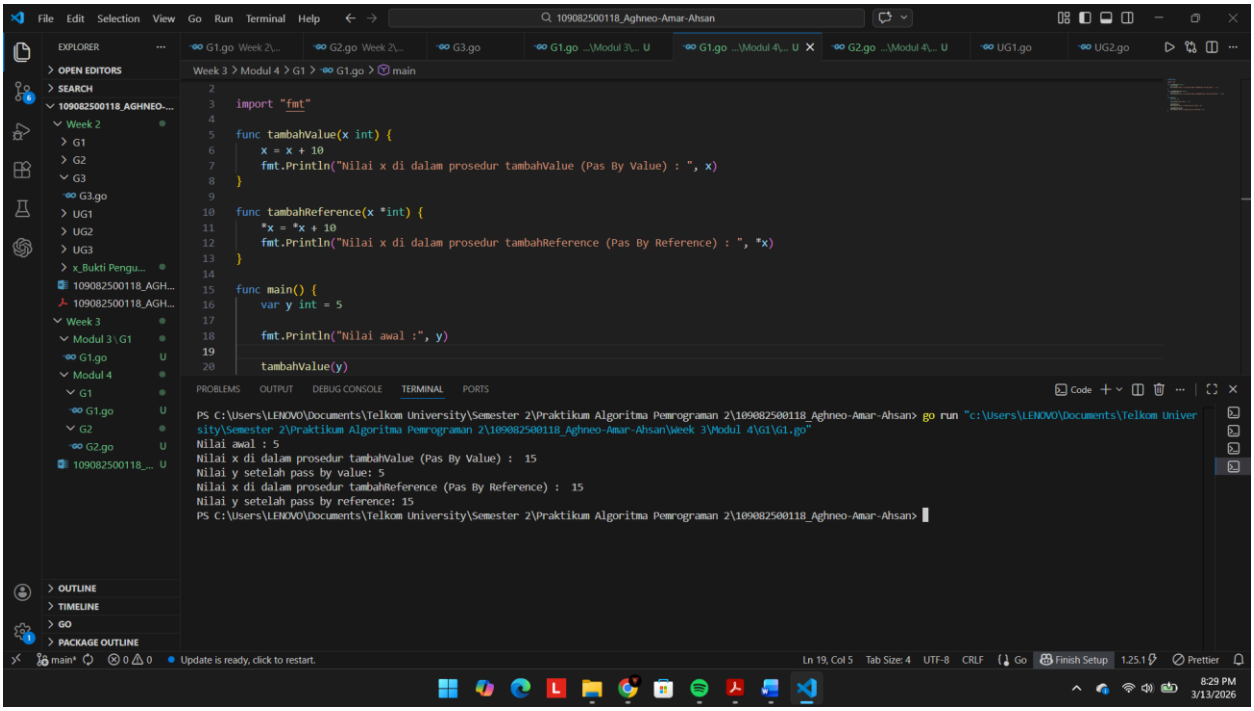


2. tambahValue dan tambahReference

```
package main

import "fmt"

func tambahValue(x int) {
    x = x + 10
    fmt.Println("Nilai x di dalam prosedur tambahValue (Pas
By Value) : ", x)
}

func tambahReference(x *int) {
    *x = *x + 10
    fmt.Println("Nilai x di dalam prosedur tambahReference
(Pas By Reference) : ", *x)
}

func main() {
    var y int = 5

    fmt.Println("Nilai awal :", y)

    tambahValue(y)
    fmt.Println("Nilai y setelah pass by value:", y)

    tambahReference(&y)
    fmt.Println("Nilai y setelah pass by reference:", y)
```

```
}
```

Output :

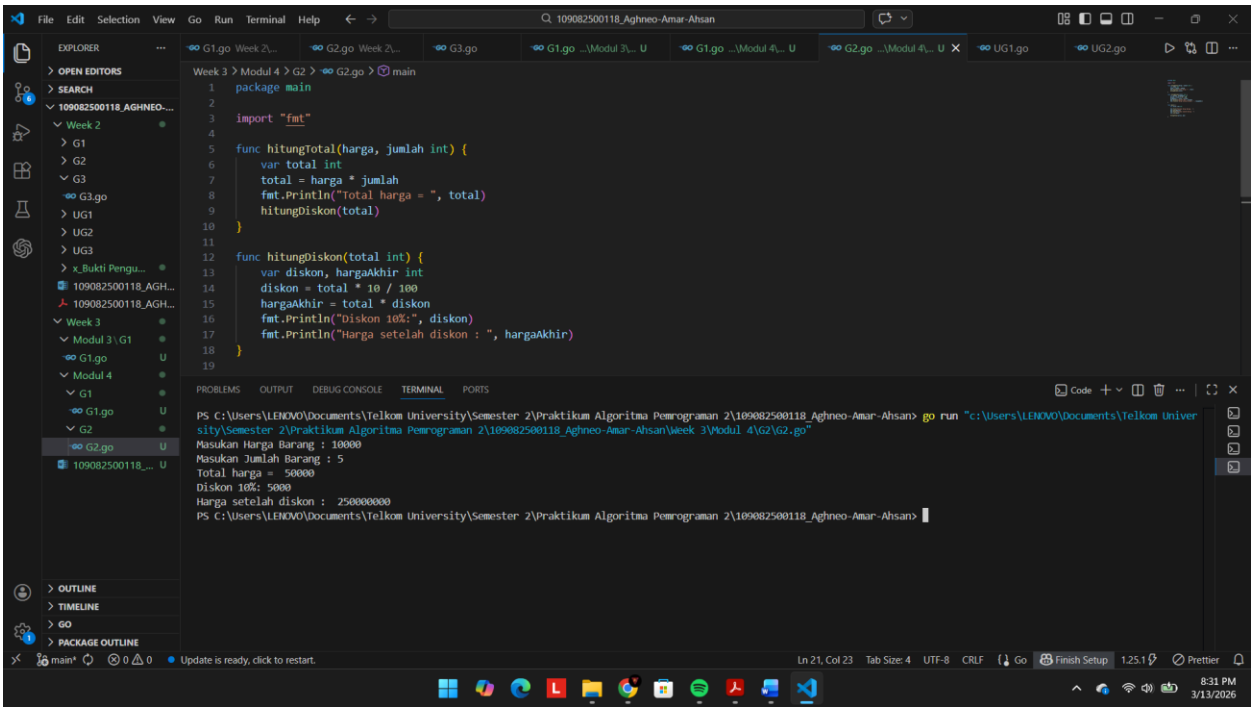


3. Diskon

```
package main

import "fmt"

func hitungTotal(harga, jumlah int) {
    var total int
    total = harga * jumlah
    fmt.Println("Total harga = ", total)
    hitungDiskon(total)
}

func hitungDiskon(total int) {
    var diskon, hargaAkhir int
    diskon = total * 10 / 100
    hargaAkhir = total * diskon
    fmt.Println("Diskon 10%:", diskon)
    fmt.Println("Harga setelah diskon : ", hargaAkhir)
}

func main() {
    var price, qty int
```

```
    fmt.Print("Masukan Harga Barang : ")
    fmt.Scan(&price)
    fmt.Print("Masukan Jumlah Barang : ")
    fmt.Scan(&qty)

    hitungTotal(price, qty)
}
```

Output :



C. Unguided

1. Permutasi dan Kombinasi

1. Minggu ini, mahasiswa Fakultas Informatika mendapatkan tugas dari mata kuliah matematika diskrit untuk mempelajari kombinasi dan permutasi. Jonas salah seorang mahasiswa, iseng untuk mengimplementasikannya ke dalam suatu program. Oleh karena itu bersediakah kalian membantu Jonas? (tidak tentunya ya :p)
   - **Input :** terdiri dari empat buah bilangan asli a, b, c, d yang dipisahkan oleh spasi, dengan syarat $a \geq c$ dan $b \geq d$
   - **Output :** terdiri dari dua baris. Baris pertama adalah hasil permutasi dan kombinasi a terhadap c, sedangkan baris kedua adalah hasil permutasi dan kombinasi b terhadap d.

   Catatan Permutasi (P) dan kombinasi (C) dari n terhadap r ($n \geq r$) dapat dihitung dengan menggunakan persamaan berikut!

   $$P(n,r) = \frac{n!}{(n-r)!}, \text{ sedangkan } C(n,r) = \frac{n!}{r!(n-r)!}$$

   Contoh :

No	Input	Output	Penjelasan
1	5 10 3 10	60 10 3628800 1	P(5,3) = 5!/2! = 120/2 = 60 C(5,3) = 5!/(3!x2!) = 120/12 = 10 P(10,10) = 10!/0! = 3628800/1 = 3628800 C(10,10) = 10!/(10!x0!) = 10!/10! = 1
2	8 0 2 0	56 28 1 1	

   Selesaikan program tersebut dengan memanfaatkan subprogram yang diberikan dibawah ini :

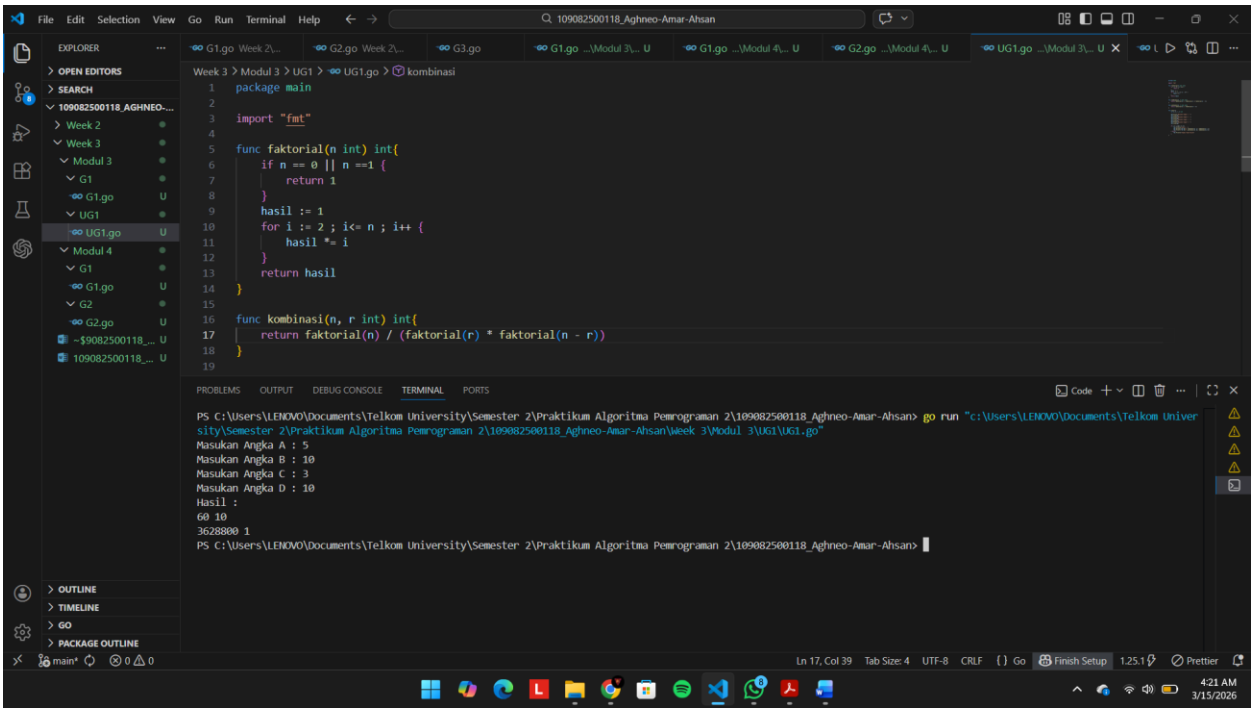
```
Function factorial (n : integer) → integer
{I.S. terdefinisi bilangan bulat positif n
 F.S. mengembalikan nilai faktorial dari n}

Function permutation(n, r : integer) → integer
{I.S. terdefinisi bilangan bulat positif n dan r, dengan n >= r
 F.S. mengembalikan hasil n permutasi r}

Function combination (n, r : integer) → integer
{I.S. terdefinisi bilangan bulat positif n dan r, dengan n >= r
 F.S. mengembalikan hasil n kombinasi r}
```

Jawaban :

```
package main

import "fmt"

func faktorial(n int) int{
    if n == 0 || n ==1 {
        return 1
    }
```

```
    hasil := 1
    for i := 2 ; i<= n ; i++ {
        hasil *= i
    }
    return hasil
}

func kombinasi(n, r int) int{
    return faktorial(n) / (faktorial(r) * faktorial(n - r))
}

func permutasi(n, r int) int{
    return faktorial(n) / (faktorial(n - r))
}

func main(){
    var a, b, c, d int

    fmt.Print("Masukan Angka A : ")
    fmt.Scan(&a)
    fmt.Print("Masukan Angka B : ")
    fmt.Scan(&b)
    fmt.Print("Masukan Angka C : ")
    fmt.Scan(&c)
    fmt.Print("Masukan Angka D : ")
    fmt.Scan(&d)

    if a >= c && b >= d {
        fmt.Println("Hasil : ")
        fmt.Printf("%d %d\n", permutasi(a, c), kombinasi(a,
c))
        fmt.Printf("%d %d\n", permutasi(b, d), kombinasi(b,
d))
    } else {
        fmt.Println("Angka Tidak Valid")
    }
}
```

Output :



Penjelasan :

Program ini ibarat kalkulator pintar untuk menghitung berapa banyak cara kita bisa menyusun atau memilih suatu barang berdasarkan empat angka masukan (A, B, C, dan D). Di belakang layar, program ini sudah dibekali tiga rumus matematika dasar untuk memproses hitungannya: faktorial (perkalian menurun), permutasi (cara menyusun dengan memperhatikan urutan), dan kombinasi (cara memilih secara acak tanpa peduli urutan). Saat kita memasukkan angka, program akan mengecek dulu apakah permintaannya sudah sesuai persyaratan atau belum. Jika syarat terpenuhi, kalkulator akan langsung memunculkan hasil perhitungan permutasi dan kombinasinya. Sebaliknya, program akan otomatis menolak dengan pesan "Angka Tidak Valid".

2. Parkir

2. Suatu perpustakaan + cafe beroperasi selama 24 jam dan memiliki sistem parkir kendaraan. Tarif parkir dibedakan berdasarkan jenis kendaraan dan rentang waktu parkir.
   - Tarif parkir motor :
     Diatas jam 12 malam – 5 sore : Rp4.000/jam
     Diatas jam 5 sore – jam 12 malam : Rp5.000/jam
   - Tarif parkir mobil :
     Diatas jam 12 malam – 5 sore : Rp6.000/jam
     Diatas jam 5 sore – jam 12 malam : Rp7.000/jam

   Tugasmu adalah membuat program yang harus :

   a. Menerima input jenis kendaraan (string), jam masuk (integer), dan jam keluar (integer).
   b. Menghitung biaya parkir kendaraan menggunakan fungsi.
   c. Program akan terus menerima input kendaraan selama masih ada kendaraan yang parkir.
   d. Jika kendaraan sudah habis, masukkan tanda "-" pada jenis kendaraan untuk menghentikan input.
   e. Setelah selesai, program menampilkan total pendapatan parkir dalam satu hari.

   Catatan :

   - Gunakan format waktu 0–24 jam.
   - Pukul 12 malam ditulis 24, bukan 00.
   - Gunakan implementasi percabangan (if-else) dan perulangan (for loop).
   - Wajib menggunakan fungsi.

   Ekspektasi output :

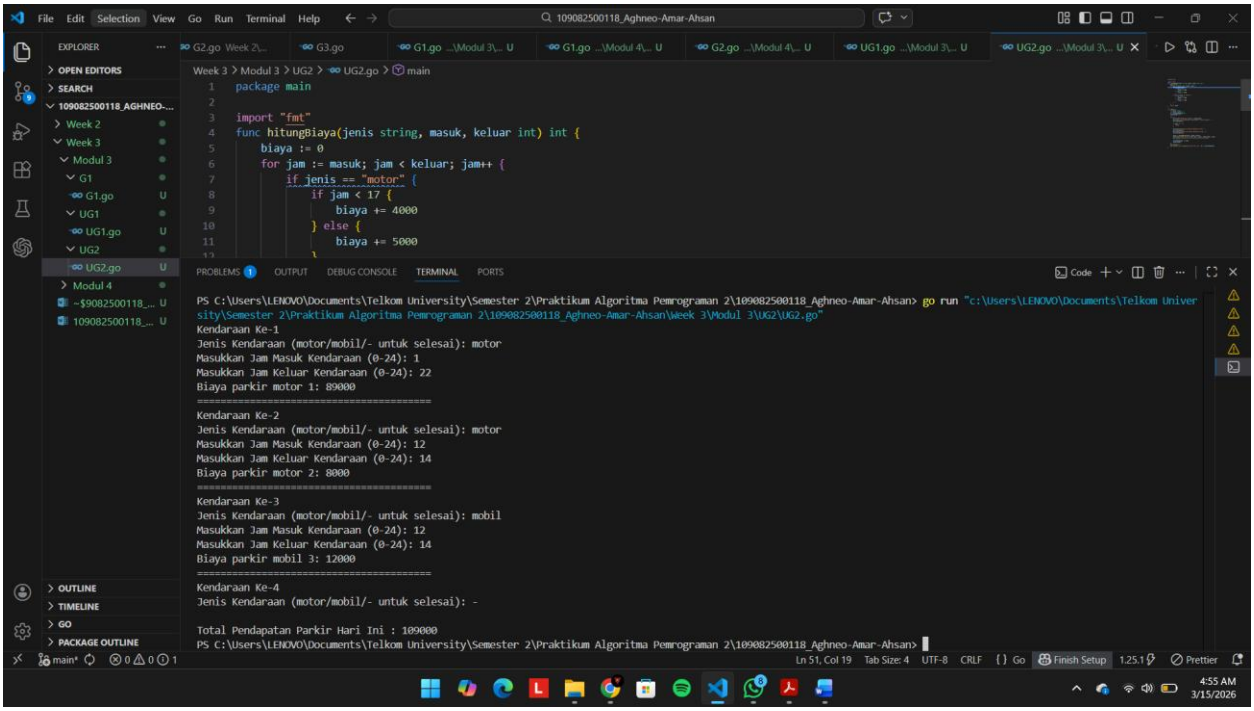
```
==== Rekap Tarif Parkir Cafe Per Hari ====

*Kendaraan 1
Kendaraan apa? (motor/mobil/- untuk selesai): motor
Masukkan Jam Masuk Kendaraan (0-24): 1
Masukkan Jam Keluar Kendaraan (0-24): 22
Biaya parkir motor 1: 89000
========================================

*Kendaraan 2
Kendaraan apa? (motor/mobil/- untuk selesai): motor
Masukkan Jam Masuk Kendaraan (0-24): 12
Masukkan Jam Keluar Kendaraan (0-24): 14
Biaya parkir motor 2: 8000
========================================

*Kendaraan 3
Kendaraan apa? (motor/mobil/- untuk selesai): mobil
Masukkan Jam Masuk Kendaraan (0-24): 12
Masukkan Jam Keluar Kendaraan (0-24): 14
Biaya parkir mobil 3: 12000
========================================

*Kendaraan 4
Kendaraan apa? (motor/mobil/- untuk selesai): -

*** Total Pendapatan Parkir Hari Ini Adalah 109000 ***
```

Jawaban :

```
package main

import "fmt"
func hitungBiaya(jenis string, masuk, keluar int) int {
    biaya := 0
    for jam := masuk; jam < keluar; jam++ {
        if jenis == "motor" {
            if jam < 17 {
                biaya += 4000
            } else {
                biaya += 5000
            }
        } else if jenis == "mobil" {
            if jam < 17 {
                biaya += 6000
```

```
            } else {
                biaya += 7000
            }
        }
    }
    return biaya
}

func main() {
    var jenis string
    var masuk, keluar int
    var totalPendapatan int
    kendaraanKe := 1

    for {
        fmt.Printf("Kendaraan Ke-%d\n", kendaraanKe)
        fmt.Print("Jenis Kendaraan (motor/mobil/- untuk
selesai): ")
        fmt.Scan(&jenis)

        if jenis == "-" {
            break
        }

        fmt.Print("Masukkan Jam Masuk Kendaraan (0-24): ")
        fmt.Scan(&masuk)
        fmt.Print("Masukkan Jam Keluar Kendaraan (0-24): ")
        fmt.Scan(&keluar)

        biaya := hitungBiaya(jenis, masuk, keluar)
        fmt.Printf("Biaya parkir %s %d: %d\n", jenis,
kendaraanKe, biaya)
        fmt.Println("=====================================
=")

        totalPendapatan += biaya
        kendaraanKe++
    }
    fmt.Println(" ")
    fmt.Printf("Total Pendapatan Parkir Hari Ini : %d",
totalPendapatan)
}
```

Output :



Penjelasan :

Program ini sangat mirip dengan sistem kasir otomatis di gerbang parkir yang bertugas menghitung tarif harian. Cara kerjanya, program akan terus-menerus meminta kita memasukkan data kendaraan baru sampai kita mengetikkan tanda strip ("-") untuk menutup sistem. Nantinya pemrosesan tarif per jam untuk setiap kendaraannya akan terjadi, di mana aturannya membedakan harga berdasarkan jenis kendaraan dan waktu. Ketika parkir sebelum jam 5 sore (17:00), maka akan dihitung lebih murah dibandingkan jam-jam setelahnya. Setiap kali satu kendaraan selesai dihitung, biayanya akan langsung ditampilkan di layar dan secara otomatis ditambahkan ke dalam total pendapatan, sehingga ketika sistem akhirnya ditutup, program akan langsung menyajikan rangkuman keseluruhan uang parkir yang berhasil terkumpul pada hari tersebut.

3. Cetak Deret

1. Skiena dan Revilla dalam Programming Challenges mendefinisikan sebuah deret bilangan. Deret dimulai dengan sebuah bilangan bulat n. Jika bilangan n saat itu genap, maka suku berikutnya adalah ½n, tetapi jika ganjil maka suku berikutnya bernilai 3n+1. Rumus yang sama digunakan terus menerus untuk mencari suku berikutnya. Deret berakhir ketika suku terakhir bernilai 1. Sebagai contoh jika dimulai dengan n = 22, maka deret bilangan yang diperoleh adalah:
   22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
   Untuk suku awal sampai dengan 1000000, diketahui deret selalu mencapai suku dengan nilai 1.
   Buatlah program **skiena** yang akan mencetak setiap suku dari deret yang dijelaskan di atas untuk nilai suku awal yang diberikan. Pencetakan deret harus dibuat dalam prosedur cetakDeret yang mempunyai 1 parameter formal, yaitu nilai dari suku awal.
   ```
   procedure cetakDeret(in n : integer)
   ```

Masukan berupa satu bilangan integer positif yang lebih kecil dari 1000000.
Keluaran terdiri dari satu baris saja. Setiap suku dari deret tersebut dicetak dalam baris yang dipisahkan oleh sebuah spasi.

No	Masukan	Keluaran
1	22	22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
2	36	36 18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
3	60	60 30 15 46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1

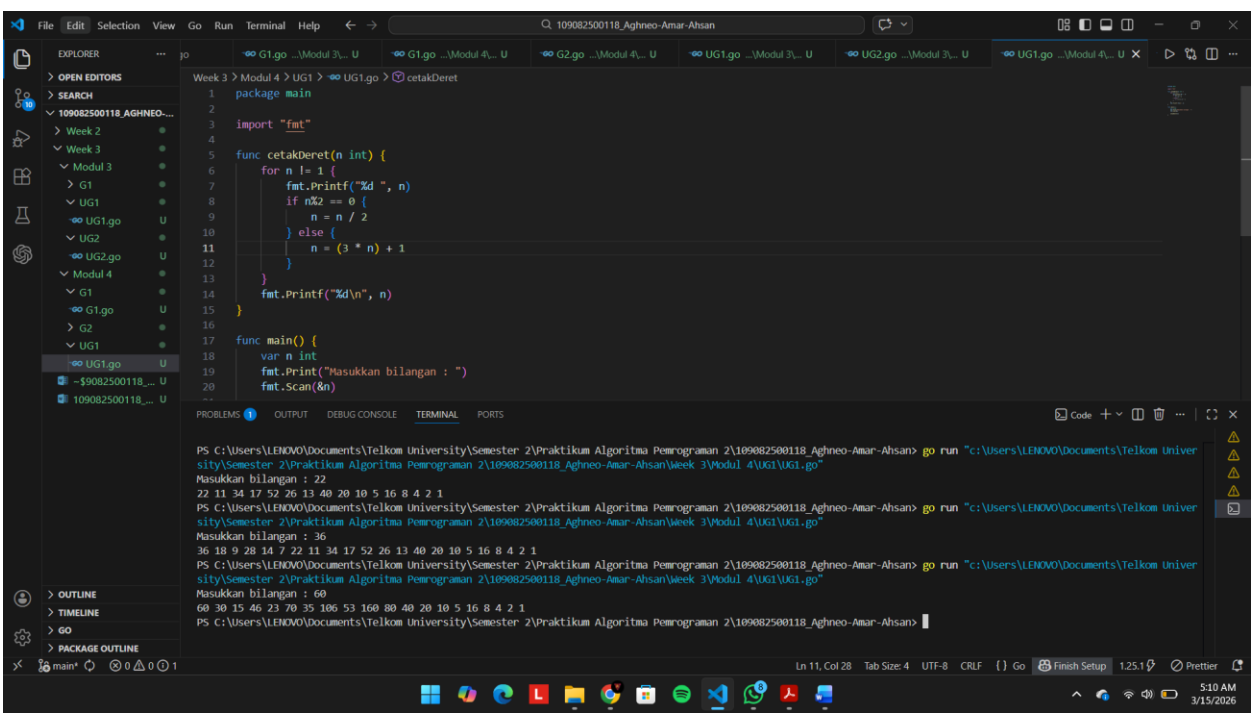
Ekspektasi Output :

```
311102151_Muhammad-Dhimas-Hafizh-Fathurrahma\Week 3 - Modul 3 & 4\Unguided\unguided-no3\no3.go"
Masukkan bilangan : 22
22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
Masukkan bilangan : 36
36 18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
Masukkan bilangan : 60
60 30 15 46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1
```

Jawaban :

```go
package main

import "fmt"

func cetakDeret(n int) {
    for n != 1 {
        fmt.Printf("%d ", n)
        if n%2 == 0 {
            n = n / 2
        } else {
            n = (3 * n) + 1
        }
    }
    fmt.Printf("%d\n", n)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan : ")
    fmt.Scan(&n)

    cetakDeret(n)
}
```

Output :



Penjelasan :
Program Go akan menghasilkan deret bilangan beruntun yang diinisiasi oleh Skiena dan Revilla. Untuk cara kerjanya, setelah kita memasukkan sebuah angka, angka tersebut akan diserahkan ke sebuah prosedur (subprogram khusus) bernama cetakDeret untuk mulai diproses. Di dalam prosedur inilah perulangan berjalan secara otomatis dengan sebuah aturan. Langkah-langkah aturan, yang pertama mesin akan mengecek angka saat itu, jika angkanya genap maka akan dibagi dua, namun jika ganjil maka akan dikalikan tiga lalu ditambah satu (3n+1). Rumus ini akan terus diterapkan berulang kali untuk mencari suku berikutnya sambil terus mencetak rentetan angkanya ke layar. Proses perhitungan ini akan terus berjalan menyusut dan baru akan benar-benar berhenti ketika deret bilangannya sudah menyentuh angka 1 sebagai titik akhir.

4. Luas dan Keliling

2. Buatlah sebuah program yang dapat menghitung luas & keliling persegi, persegi panjang, dan lingkaran. Setiap operasi perhitungan bangun datar dibuat prosedurnya sendiri-sendiri :

```
Procedure hitungPersegi(sisi : integer)
{I.S. terdefinisi bilangan bulat sisi
 F.S. menampilkan hasil perhitungan luas & keliling persegi}

Procedure hitungPersegiPanjang(panjang, lebar : integer)
{I.S. terdefinisi bilangan bulan Panjang & lebar
 F.S. menampilkan hasil perhitungan luas & keliling persegi
Panjang}

Procedure hitungLingkaran(jarijari : float64)
{I.S. terdifinisi bilangan decimal jarijari
 F.S. menampilkan hasil perhitungan luas & keliling lingkaran}
```

Buat menu tampilan awal program seperti berikut ini. (hint : gunakan percabangan switch-case)

```
--- PROGRAM BANGUN DATAR ---
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan : 
```

Kemudian hitunglah luas & keliling :

1) Persegi dengan sisi 1057
2) Persegi Panjang dengan Panjang 106 dan lebar 88
3) Lingkaran dengan jari-jari 13,87

Ekspektasi output :

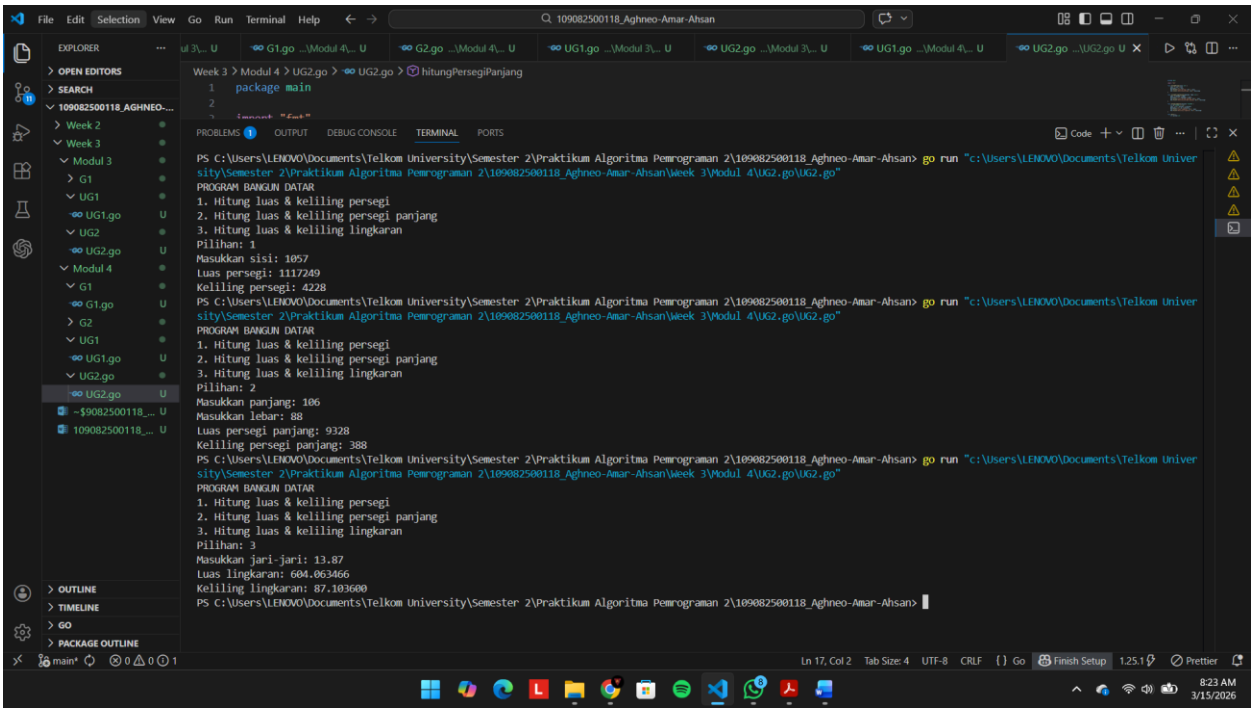
```
PS D:\File Dhimas\Asprak\Asprak Semester 6\Alpro 2\2311102151_Muhammad-Dhimas-Hafizh-Fathurrahmao> go run
--- PROGRAM BANGUN DATAR ---
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan : 1

Masukkan sisi : 1057
Luas persegi :  1117249
Keliling persegi :  4228
--- PROGRAM BANGUN DATAR ---
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan : 2

Masukkan panjang : 106
Masukkan lebar : 88
Luas persegi panjang :  9328
Keliling persegi panjang :  388
--- PROGRAM BANGUN DATAR ---
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan : 3

Masukkan jari-jari : 13.87
Luas lingkaran :  604.063466
Keliling lingkaran :  87.3036
PS D:\File Dhimas\Asprak\Asprak Semester 6\Alpro 2\2311102151_Muhammad-Dhimas-Hafizh-Fathurrahmao>
```

Jawaban :

```go
package main

import "fmt"

func hitungPersegi(sisi int) {
    luas := sisi * sisi
    keliling := 4 * sisi
    fmt.Printf("Luas persegi: %d\n", luas)
    fmt.Printf("Keliling persegi: %d\n", keliling)
}

func hitungPersegiPanjang(panjang, lebar int) {
    luas := panjang * lebar
    keliling := 2 * (panjang + lebar)
    fmt.Printf("Luas persegi panjang: %d\n", luas)
    fmt.Printf("Keliling persegi panjang: %d\n", keliling)
}

func hitungLingkaran(jarijari float64) {
    var pi float64 = 3.14
    luas := pi * jarijari * jarijari
    keliling := 2 * pi * jarijari
    fmt.Printf("Luas lingkaran: %f\n", luas)
    fmt.Printf("Keliling lingkaran: %f\n", keliling)
}
```

```go
func main() {
    var pilihan int

    fmt.Println("PROGRAM BANGUN DATAR")
    fmt.Println("1. Hitung luas & keliling persegi")
    fmt.Println("2. Hitung luas & keliling persegi panjang")
    fmt.Println("3. Hitung luas & keliling lingkaran")
    fmt.Print("Pilihan: ")
    fmt.Scan(&pilihan)

    switch pilihan {
    case 1:
        var sisi int
        fmt.Print("Masukkan sisi: ")
        fmt.Scan(&sisi)
        hitungPersegi(sisi)
    case 2:
        var panjang, lebar int
        fmt.Print("Masukkan panjang: ")
        fmt.Scan(&panjang)
        fmt.Print("Masukkan lebar: ")
        fmt.Scan(&lebar)
        hitungPersegiPanjang(panjang, lebar)
    case 3:
        var r float64
        fmt.Print("Masukkan jari-jari: ")
        fmt.Scan(&r)
        hitungLingkaran(r)
    default:
        fmt.Println("Pilihan yang dimasukkan tidak valid.")
    }
}
```

Output :



Penjelasan :

Program bertujuan untuk menghitung luas dan keliling tiga jenis bangun datar, yaitu persegi, persegi panjang, dan lingkaran. Saat dijalankan, program akan langsung menyajikan daftar pilihan menu dengan menggunakan case. Program ini disusun dengan rapi karena menggunakan tiga prosedur terpisah yang mana tujuan prosedur memang untuk memudahkan dan membuat tampilan kode menjadi lebih rapih. Begitu kita memilih menu dan memasukkan angka ukurannya, sistem akan memanggil jenis prosedur yang tepat untuk mengolah rumus matematikanya, lalu posedur tersebut akan langsung mencetak hasil perhitungan luas beserta kelilingnya sebagai output akhir.

D. Kesimpulan

Kesimpulannya, penerapan materi Function dan Procedure ini sangat membantu dalam membuat kode program kita menjadi jauh lebih rapi dan terstruktur. Perbedaannya cukup sederhana yaitu function dapat diibaratkan sebagai sebuah proses yang mengolah data dan menghasilkan suatu nilai akhir (memiliki nilai return), sedangkan procedure lebih ditujukan untuk memberikan instruksi kepada program agar menjalankan tugas tertentu tanpa mengembalikan nilai apa pun. Dengan memecah logika program yang panjang ke dalam beberapa subprogram kecil seperti ini, kita dapat menghindari penulisan sintaks yang berulang-ulang, sehingga kode cukup dipanggil kembali saat dibutuhkan. Pada akhirnya, isi program utama kita akan terlihat jauh lebih bersih, alurnya lebih mudah dipahami, dan tentunya akan sangat mempermudah proses pencarian kesalahan atau bug nantinya.